

Design and Analysis of Algorithms

Text Book :

Fundamentals of Computer Algorithms, Ellis Horowitz,
Satraj sahani and Rajashekaram, Galgotia publications pvt.
Ltd.

Unit-I

Introduction:

Algorithm, Pseudo code for expressing algorithms, Performance An Algorithm, Pseudo code for expressing algorithms, Performance Analysis-Space complexity, Time complexity, **Asymptotic Notation-** Big oh notation, Omega notation, Theta notation and Little oh notation, Probabilistic Analysis, Connected and Bi-connected Components.

Algorithm

- Definition

An *algorithm* is a set of steps used to solve a specific problem.

➤ The word algorithm comes from the name of the person author

-Abu Jafar Mohammed Ibn Musa Al khowarizmi who wrote a text book entitled-"Algorithmi de numero indorum" Now term" Algorithmi "in the title of the book led to the term Algorithm.

➤ An algorithm is an effective method for finding out the solution for a given problem. It is a sequence of instruction
That conveys the method to address a problem

➤ **Algorithm** : Step by step procedure to solve a computational problem is called Algorithm.

INTRODUCTION

- An algorithm is a set of steps of operations to solve a problem performing calculation, data processing, and automated reasoning tasks.
- An algorithm is an efficient method that can be expressed within finite amount of Time and space.
- The important aspects of algorithm design include creating an efficient algorithm to solve a problem in an efficient way using minimum time and space.
- To solve a problem, different approaches can be followed. Some of them can be efficient with respect to time consumption, whereas other approaches may be memory efficient.

PROPERTIES OF ALGORITHM

TO EVALUATE AN ALGORITHM WE HAVE TO SATISFY THE FOLLOWING CRITERIA:

1. **INPUT:** The Algorithm should be given **zero** or more input.
2. **OUTPUT:** **At least one** quantity is produced. For each input the algorithm produces a value from specific task.
3. **DEFINITENESS:** Each instruction is clear and **unambiguous**.
4. **FINITENESS:** If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after a **finite number** of steps.
5. **EFFECTIVENESS:** Every instruction must **very basic** so that it can be carried out, in principle, by a person using only pencil & paper.

Pseudo code for expressing algorithms

We present most of our algorithms using pseudo code that looks like C and Pascal code.

1. Comments begin with `//` and continue until end of the line.
2. Blocks are indicated with matching braces: `{` and `}`.
 - i. A compound statement.
 - ii. Body of a function.

3.

- i). The data types of variables are not explicitly declared.
- ii). The types will be clear from the context.
- iii). Whether a variable is global or local to a function will also be clear from the context.

4. Assignment statement

$\langle \text{variable} \rangle := \langle \text{expression} \rangle$

5. Boolean values are **true** and **false**.

Logical operators are **and**, **or** and **not** R

Relational operators are $<$, $\leq$, $=$, $\neq$, $\geq$ and $>$

6. Elements of multidimensional arrays are accessed using [and]. For example the (i,j)th element of the array A is denoted as $A[i,j]$.

7. Looping statements are **for**, **while**, and **repeat until**.

The general form of a **while** loop:

```
while( condition ) do  
{  
    statement_1;  
    :  
    statement_n;  
}
```

The general form of a **for** loop:

```
for variable := value1 to value2 step step do  
{  
    statement_1;  
    :  
    statement_n;  
}
```

- Here *value1*, *value2*, and *step* are arithmetic expressions.
- The clause “**step** *step*” is optional and taken as +1 if it does not occur.
- *step* could be either positive or negative.

Ex: 1: for *i*:= 1 to 10 step 2 do // increment by 2, 5 iterations
 2: for *i*:= 1 to 10 do // increment by 1, 10 iterations

- The general form of a **repeat-until** loop:

repeat

<statement 1>

:

<statement n>

until (condition)

- The statements are executed as long as condition is false.

Ex: sum:=0, number:=10;

repeat

sum := sum + number;

number := number - 1;

until **number = 0;**

8. A conditional statements

if < condition > **then** < statement >

if < condition > **then** < statement 1 > else
 < statement 2 >

9. Input and output are done using the instructions **read** and **write**.

10. Procedure or function starts with the word
Algorithm.

General form :

```
Algorithm Name( <parameter list> )  
{  
    body  
}
```

where *Name* is the name of the procedure.

- Simple variables to functions are passed by value.
- Arrays and records are passed by reference.

Ex:- Algorithm that finds and returns the maximum of n given numbers.

Algorithm max(a,n)

// a is an array of size n

{

 result:=a[1];

 for i:=2 to n do

 if (a[i] > result) then

 result:=a[i];

return result;

}

Ex:- Write an algorithm to find sum of n numbers

Algorithm sum(a, n)

// a is an array of size n

```
{
    sum:=0;
    for i :=1 to n do
    {
        sum := sum+a[i];
    }
    return sum;
}
```

How To Write an Algorithm

Step-1:start

Step-2:Read a,b,c

Step-3:if a>b

 if a>c

 print a is largest

 else

 if b>c

 print b is largest

 else

 print c is largest

Step-4 : stop

Step-1: start

Step-2: Read a,b,c

Step-3:if a>b then go to step 4

 otherwise go to step 5

Step-4:if a>c then

 print a is largest otherwise

 print c is largest

Step-5: if b>c then

 print b is largest otherwise

 print c is largest

step-6: stop

Differences

Algorithm

1. At design phase phase
2. Natural language
3. Person should have Domain knowledge
4. Analyze

Program

1. At Implementation
2. written in any programming language
3. Programmer
4. Testing

Space Complexity

- The space needed by an algorithm is called a space complexity.
- It has the following components.
 - Instruction space / Program space
 - It does not depend on the number of inputs. (Instance characteristics of a problem).
 - Data space
 - *Constants & simple variables* – Don't depend on the number of inputs.
 - *Dynamically allocated objects* - Depends on the number of inputs.
 - *Stack space* - Generally does not depend on the number of inputs unless recursive functions are in use.

Recursive algorithm

Algorithm rfactorial(n)

// n is an integer

```
{      fact=1;
      if(n=1 or n=0) return fact;
      else
        fact=n*rfactorial(n-1);
      return fact;
}
```

Note : Each time the recursive function is called, the current values of **n**, **fact** and the **address of the next statement** to return to on completion are saved on the stack.

Execution

R:

```
void main()
{int
R=rfactorial(3);
printf(R);}
```

n : 3

```
int rfactorial(n){
    fact=1;
    if (n=0 or n=1)return fact;
    else
    fact=n*rfactorial(n-1);
    return fact;
}
```

n : 2

```
int rfactorial(n){
    fact=1
    if (n=0 or n=1)return fact
    else
    fact=n*rfactorial(n-1);
    return fact;
}
```

n : 1

```
int rfactorial(n){
    if (n=0 or n=1)
        return 1;
    else
        fact=n*rfactorial(n-1);
    return fact;
}
```

location to return

fact
n

1
1

fact=1

4000

fact=2*

1
2

fact
n

3000

fact=3*

1
3

fact
n

2000

R

Stack

Contd..

Therefore, We can divide the total space required by a program into two parts:

i) Fixed Space Requirements (C)

Independent of the characteristics of the problem instance (I)

- Instruction space
- Space for simple variables and constants.

ii) Variable Space Requirements ($S_p(I)$)

depend on the characteristics of the problem instance (I)

- *Dynamically allocated objects* (Number of inputs associated with I)
- Recursive stack space (formal parameters, local variables, return address).

– Therefore, the space requirement of any problem P can be written as

$$S(p) = C + S_p(\text{Instance characteristics}).$$

Note:

- We concentrate only on estimating S_p (Instance characteristics).
- We don't concentrate on estimating fixed part c .
- We need to identify the instance characteristics of the problem to measure S_p

Example 1

Algorithm $abc(a,b,c)$

```
{  
    return  $a+b+b*c+(a+b-c)/(a+b)+4.0$ ;  
}
```

- Problem instance characterized by the specific values of a, b , and c .
- If *we assume* one word (4 bytes) is adequate to store the values of each a, b , and c , then the space required by a, b, c is independent of the instance characteristics.

Therefore, $S_{abc}(\text{instance characteristics})=0$

Example2

Algorithm sum(a,n)

```
{  
    s:=0;  
    for i:=1 to n do  
        s:=s+a[i];  
    return s;  
}
```

- Problem instance characterized by n.
- The amount of space depends on the value of n.

Therefore, $S_{\text{sum}}(n) = n$

Example3

```
Algorithm RSum(a,n)
{
    if( $n \leq 0$ ) then return 0;
    else return RSum(a,n-1)+a[n];
}
```

Type	Name	Number of bytes
formal parameter: int	a	2
formal parameter: int	n	2
return address (used internally)		2
Total per one recursive call		6

Total no.of recursive calls n, therefore **$S_{\text{RSum}}(n)=6n$**

Contd..

- Time Complexity:

$$T(P) = C + T_p(I)$$

- Time Complexity, $T(P)$ is the sum of its compile time C plus its run time, $T_p(I)$.
- Compile time does not depend on the instance characteristics(*Number of inputs*).
- We will concentrate on estimating run time $T_p(I)$.

- If we know the compiler used, then we can find out the
 - No. of additions , subtractions, and so on.
- Then we can obtain an expression for $T_p(n)$ Of the form

$$T_p(n) = c_a ADD(n) + c_s SUB(n) + c_m MUL(n) + c_d DIV(n) + \dots$$

where,

- c_a , c_s , c_m , c_d and so on denote the time required for an addition , subtraction and so on.
- ADD, SUB, MUL, DIV, and so denote no.of additions, subtractions, and so on.

- Even with this formula it is not possible to calculate the exact execution time, because time required for an addition, subtraction, and so on, depends on numbers being added, subtracted, and so on.
- So, the value of $T_p(n)$ for any given n can be obtained only experimentally (using stopwatch).
- Even with the experimental approach, we face difficulties.
- The execution time of a program p depends on the number of other programs running on the computer at the time program p is running.

Contd..

- As there are some problems in finding the execution time using earlier methods, we will go one step further and count only the number of *program steps*.

program step

program step is *loosely* defined as a syntactically or semantically meaningful program *statement* whose execution time is independent of the *number of inputs*.

Example :

$c := a + b;$

$\text{sum} := \text{sum} + a[i];$

- The number of steps contributed by any *program statement* depends on the kind of statement.
- **Comments** are counted as **zero** number of steps.
- An **assignment** statement which does not involve any function calls to other functions counted as **one** step.
- For loops, such as the **for**, **while**, and **repeat-until**, we consider the step counts only for the **control part** of the statement.
- The control parts for **for** and **while** statements have the following forms:
 - for i:= <expr1> to <expr2> do
 - while (<expr>) do

- Each execution of the control part of a while statement is one, unless **<expr>** is a function of instance characteristics.
- Similarly, each execution of the control part of a **for** statement is one, unless **<expr1>** and **<expr2>** are functions of the instance characteristics.
- Each execution of the **condition** of a conditional statements step count is one, unless condition is a function of instance characteristics.
- If any statement (**assignment statement, control part, condition etc.**) involves function calls, then the step count is equal to the number of steps assignable to the **function** plus one.

- Methods to compute the step count

- 1) Introduce global variable **count** into the program with initial value zero.

- Statements to increment **count** by the appropriate amount are introduced.
- The value of the **count** at the time program terminates is the number steps contributed by the program.

- 2) Tabular method

- Determine the number of steps contributed by each statement per **execution $\times$ frequency (number of times stmt is executed)**.
- Add up the contribution of all statements

Method-I: Introduce variable **count** into programs

EX:- Iterative sum of n numbers

```
Algorithm sum(a, n)
{
    s:=0;
    count:=count+1; // for assignment statement
    for i:=1 to n do
    { count:=count+1; // For for
      s:=s+a[i];
      count:=count+1; // for assignment statement
    }
    count:=count+1; // for last time of for
    return s;
    count:=count+1; // for return
}
```

$2n + 3$ steps

Note : *Step count tells us how the run time for a program changes with changes in the instance characteristics.*

Ex:- Addition of two $m \times n$ matrices

Algorithm Add(a,b,c,,m,n)

```
{  
  for i:=1 to m do  
  {  
    for j:=1 to n do  
    {  
      c[i,j]:=a[i,j]+b[i,j];  
    }  
  }  
}
```

$2mn + 2m + 1$ steps

EX:- Recursive sum of n numbers

Algorithm RSum(a,n)

```
{
    count:=count+1; // for the if conditional
    if(n ≤ 0) then
    {
        return 0;
        count:=count+1; // for the return
    }
    else
    {
        return RSum(a,n-1)+a[n];
        count:=count+1+x; // For the addition, function invocation and return
        // x = tRSum(n-1)
    }
}
```

- When analyzing a recursive program for its step count, we generally obtain a recursive formula for the step count.
- We obtain the following recursive formula for above (RSum) algorithm.

$$t_{\text{RSum}}(n) = \begin{cases} 2 & \text{If } n=0 \\ 2 + t_{\text{RSum}}(n-1) & \text{If } n>0 \end{cases}$$

- One of the ways of solving such recursive formula is by repeated substitutions for each occurrence of the function t_{RSum} on the right side until all such occurrences disappear:

$$\begin{aligned}t_{\text{RSum}}(n) &= 2 + t_{\text{RSum}}(n-1) \\&= 2 + 2 + t_{\text{RSum}}(n-2) \\&= 2(2) + t_{\text{RSum}}(n-2) \\&\vdots \\&\vdots \\&= n(2) + t_{\text{RSum}}(n-n) \\&= 2n + t_{\text{RSum}}(0) \\&= 2n + 2\end{aligned}$$

The step count for Rsum is $2n+2$

Ex : Find out total step count of the following algorithm using method 1

```
Algorithm display( )
{
  for i := 1 to n do
  {
    for j := 1 to n do
    {
      write ( i, j );
    }
  }
}
```

$2n^2 + 2n + 1$ steps

Method-II: Tabular method

- Ex:- Iterative sum of n numbers

Statement	s/e	frequency	Total steps
Algorithm sum(a, n)	0	--	0
{	0	--	0
s:=0 ;	1	1	1
for i:=1 to n do	1	n+1	n+1
s:=s+a[i];	1	n	n
return s;	1	1	1
}	0	--	0
Total			2n+3

- EX:- Addition of two $m \times n$ matrices

Statement	s/e	frequency	Total steps
Algorithm Add(a,b,c,m, n)	0	--	0
{	0	--	0
for i:=1 to m do	1	m+1	m+1
for j:=1 to n do	1	m(n+1)	mn+m
c[i,j]:=a[i,j]+b[i,j] ;	1	mn	mn
}	0	--	0
Total			$2mn+2m+1$

- Ex:- Recursive sum of n numbers

Statement	s/e	Frequency		Total steps	
		n=0	n>0	n=0	n>0
Algorithm RSum(a,n)	0	--	--	0	0
{	0	--	--	0	0
if(n ≤ 0) then	1	1	1	1	1
return 0;	1	1	0	1	0
else return					
Rsum(a,n-1)+a[n] ;	1+x	0	1	0	1+x
}	0	--	--	0	0
Total				2	2+x

$$x = t_{\text{RSum}}(n-1)$$

Note : See slide 36 for solving this recursive formula.

Summary of Time Complexity

- The time complexity of an algorithm is given by the number of program steps taken by the algorithm.
- The number of steps is a function of the instance characteristics (number of inputs).
- Step count tells us how the run time of a program changes with changes in the number of inputs.

Best, Worst, Average Cases

- ✓ All the inputs of a given size for a given problem, don't take the same number of program steps.
- ✓ Sequential search for *Key* in an array of *n* integers:
 - Begins at the first element in array and look at each element in turn until *K* is found.

Algorithm sequentialSrch(a, n, key)

```
{  
    for i := 1 to n do  
    {  
        if ( a[ i ] = key )  
        { return i; }  
    }  
}
```

Ex : Let us take , a [] := { 2, 5, 7, 8, 9 }

Best case, key := 2

Worst case key := 9

Avg Case , if key := 2 , total no.of steps +
if key := 5 , total no.of steps
+
if key :=7, total no.of steps +
and so on by total no.of
elements (n) .

1. Best-Case Step count:-

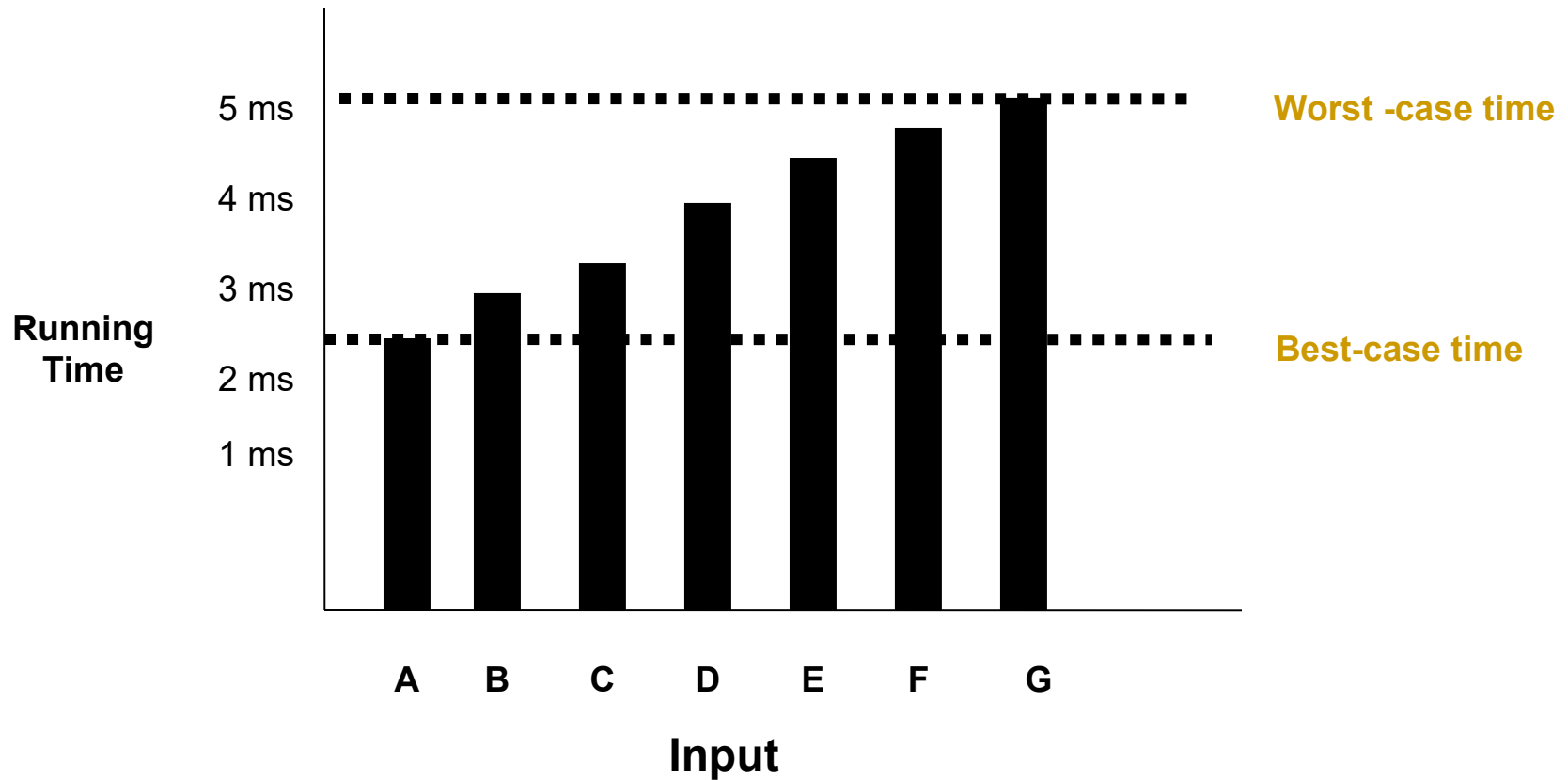
Minimum number of steps taken by the algorithm for the given inputs.

2. Worst-Case Step count:-

Maximum number of steps taken by the algorithm for the given inputs.

3. Average-Case Step count:-

Average number of steps taken by an algorithm.



$$\text{Average-case time} = \frac{\text{A time} + \text{B time} + \dots + \text{G time}}{7 \text{ (total number of elements)}}$$

Inexactness of step count.

- Both the instructions $x=y$; and $x=y+z+(x/y)$ count as one step.
- Therefore, two analysts may arrive at $4n^2+3n+2$ and $7n^2+6n+4$ as the step count for the same program.
- Because of the inexactness of a step count , the exact step count is not very useful for comparison of algorithms.

Asymptotic efficiency

- Asymptotic efficiency means study of algorithms *efficiency / time complex* for large inputs.
- To compare two algorithms with running times $f(n)$ and $g(n)$, we need a **rough measure** that characterizes **how fast each function grows** as n grows.
- *Hint:* use *rate of growth*
- Compare functions **asymptotically!**
(i.e., **for large values of n**)

Rate of Growth

- Ex:- $F(n) = n^2 + 100n + \log_{10}n + 1000$

n	f(n)	n ²		100n		log ₁₀ n		1000	
	value	Value	%	value	%	value	%	value	%
1	1,101	1	0.1	100	9.1	0	0.0	1,000	90.83
10	2,101	100	4.76	1,000	47.6	1	0.05	1,000	47.60
100	21,002	10,000	47.6	10,000	47.6	2	0.001	1,000	4.76
1,000	1,101,003	1,000,000	90.8	100,000	9.1	3	0.0003	1,000	0.09
10,000	101,001,004	100,000,000	99.0	1,000,000	0.99	4	0.0	1,000	0.001

- The low order terms and constants in a function are relatively insignificant for **large** n

$$n^2 + 100n + \log_{10}n + 1000 \sim n^2$$

i.e., we say that $n^2 + 100n + \log_{10}n + 1000$ and n^2 have the same **rate of growth**

Some more examples

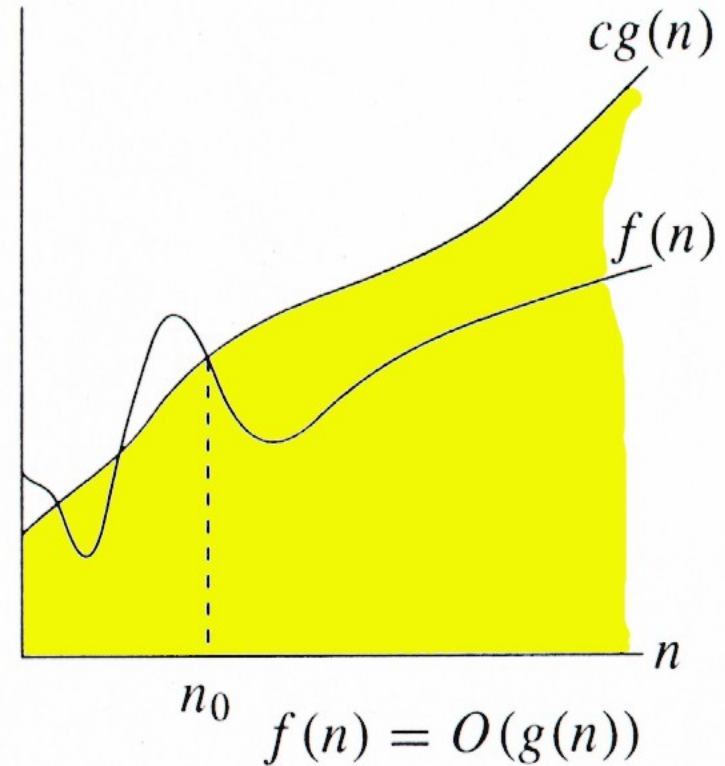
- $n^4 + 100n^2 + 10n + 50$ is $\sim n^4$
- $10n^3 + 2n^2$ is $\sim n^3$
- $n^3 - n^2$ is $\sim n^3$
- constants
 - 10 is ~ 1
 - 1273 is ~ 1

Asymptotic Notations

- Asymptotic notation describes the behavior of functions for the large inputs.
- Big Oh(O)** notation:
 - The big oh notation specifies an **upper bound** for the **growth rate** of the function **f**.

Definition: [Big “oh”]

- $f(n) = O(g(n))$ (read as “ f of n is big oh of g of n ”) iff there exist positive constants c and n_0 such that
 $f(n) \leq cg(n)$ for all $n, n \geq n_0$.



- The definition states that the function $f(n)$ is at **most** c times the function $g(n)$ except when n is smaller than n_0 .
- *In other words, $f(n)$ grows slower than or same rate as” $g(n)$.*
- When providing an upper –bound function g for f , we normally use a single term in n .

- **Examples**

- $f(n) = 3n+2$

- $3n + 2 \leq 4n$, for all $n \geq 2$, $\therefore 3n + 2 = O(n)$

- $f(n) = 10n^2+4n+2$

- $10n^2+4n+2 \leq 11n^2$, for all $n \geq 5$, $\therefore 10n^2+4n+2 = O(n^2)$

- $f(n)=6*2^n+n^2=O(2^n)$ /* $6*2^n+n^2 \leq 7*2^n$ for $n \geq 4$ */

- It also possible to write $10n^2+4n+2 = O(n^3)$ since $10n^2+4n+2 \leq 7n^3$ for $n \geq 2$
- Although n^3 is an upper bound for $10n^2+4n+2$, it is not a tight upper bound; we can find a smaller function (n^2) that satisfies big oh relation.
- But, we can not write $10n^2+4n+2 = O(n)$, since it does not satisfy the big oh relation for sufficiently large input.

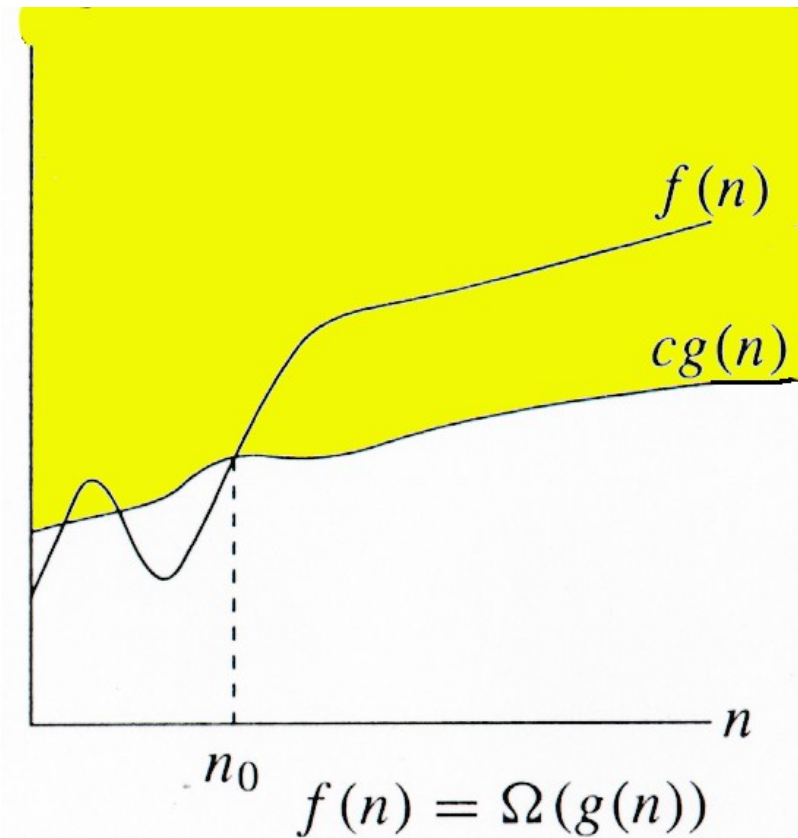
Note : *We always consider tight upper bound.*

- **Omega (Ω) notation:**

- The omega notation specifies **lower bound** for the **growth rate** of the function **f**.

Definition: [Omega]

- $f(n) = \Omega(g(n))$ (read as “ f of n is omega of g of n ”) iff there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all n ,
 $n \geq n_0$.



- The definition states that the function $f(n)$ is at **least** c times the function $g(n)$ except when n is smaller than n_0 .
- *In other words, $f(n)$ grows faster than or same rate as” $g(n)$.*

- **Examples**

- $f(n) = 3n+2$

- $3n + 2 \geq 3n$, for all $n \geq 1$, $\therefore 3n + 2 = \Omega(n)$

- $f(n) = 10n^2+4n+2$

- $10n^2+4n+2 \geq n^2$, for all $n \geq 1$, $\therefore 10n^2+4n+2 = \Omega(n^2)$

- It also possible to write $10n^2+4n+2 = \Omega(n)$ since $10n^2+4n+2 \geq n$ for $n \geq 0$

- Although n is a lower bound for $10n^2+4n+2$, it is not a tight lower bound; we can find a larger function (n^2) that satisfies omega relation.
- But, we can not write $10n^2+4n+2 = \Omega(n^3)$, since it does not satisfy the omega relation for sufficiently large input.

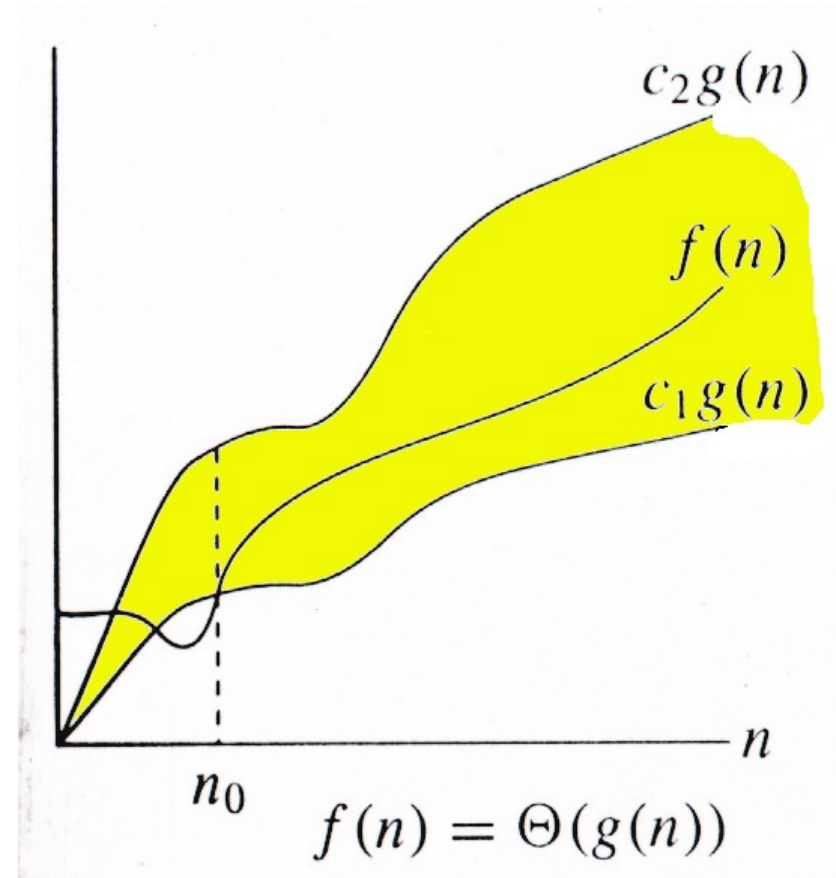
Note : We always consider tight lower bound.

- **Theta (Θ) notation:**

- The Theta notation specifies the tight upper and lower bounds for the **growth rate** of the function **f**.

Definition: [Theta]

- $f(n) = \Theta(g(n))$ (read as “ f of n is theta of g of n ”) iff there exist positive constants c_1 , c_2 , and n_0 such that $c_1g(n) \leq f(n) \leq c_2g(n)$ for all n , $n \geq n_0$.



- The definition states that the function $f(n)$ lies between c_1 times the function $g(n)$ and c_2 times the function $g(n)$ except when n is smaller than n_0 .
- In other words, $f(n)$ grows same rate as $g(n)$.
- *Examples:-*
 - $f(n) = 3n+2$
 - $3n \leq 3n + 2 \leq 4n$, for all $n \geq 2$, $\therefore 3n + 2 = \Theta(n)$
 - $f(n) = 10n^2+4n+2$
 - $n^2 \leq 10n^2+4n+2 \leq 11n^2$, for all $n \geq 5$, $\therefore 10n^2+4n+2 = \Theta(n^2)$
- But, we can not write either $10n^2+4n+2 = \Theta(n)$ or $10n^2+4n+2 = \Theta(n^3)$, since neither of these will satisfy the theta relation.

- **Little Oh(o) notation:**

- The little oh notation specifies a **strict upper bound** for the **growth rate** of the function **f**.

Definition: [Little “oh”]

- $f(n) = o(g(n))$ (read as “ f of n is little oh of g of n ”) iff

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

- The definition states that the function $f(n)$ is less than c times the function $g(n)$ except when n is smaller than n_0 .
- *In other words, $f(n)$ grows slower than" $g(n)$.*
- **Examples**

- $f(n) = 3n+2 = o(n^2)$

since $\lim_{n \rightarrow \infty} \frac{3n+2}{n^2} = 0$

- *However, $3n+2 \neq o(n)$*

Big-Oh, Theta, Omega and Little-oh

Tips :

- Think of $O(g(n))$ as “less than or equal to” $g(n)$
 - Upper bound: “grows slower than or same rate as” $g(n)$
- Think of $\Omega(g(n))$ as “greater than or equal to” $g(n)$
 - Lower bound: “grows faster than or same rate as” $g(n)$
- Think of $\Theta(g(n))$ as “equal to” $g(n)$
 - “Tight” bound: same growth rate
- Think of $o(g(n))$ as “less than to” $g(n)$
 - Strict Upper bound: “grows slower than ” $g(n)$
- (*True for large N*)

Functions ordered by growth rate

Function	Name
1	Growth is constant
$\log n$	Growth is logarithmic
n	Growth is linear
$n \log n$	Growth is $n \log n$
n^2	Growth is quadratic
n^3	Growth is cubic
2^n	Growth is exponential
$n!$	Growth is factorial

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

- To get a feel for how the various functions grow with n , you are advised to study the following facts:

		Instance characteristic n					
Time	Name	1	2	4	8	16	32
1	Constant	1	1	1	1	1	1
$\log n$	Logarithmic	0	1	2	3	4	5
n	Linear	1	2	4	8	16	32
$n \log n$	Log linear	0	2	8	24	64	160
n^2	Quadratic	1	4	16	64	256	1024
n^3	Cubic	1	8	64	512	4096	32768
2^n	Exponential	2	4	16	256	65536	4294967296
$n!$	Factorial	1	2	24	40326	20922789888000	26313×10^{33}

Figure 1 Function values

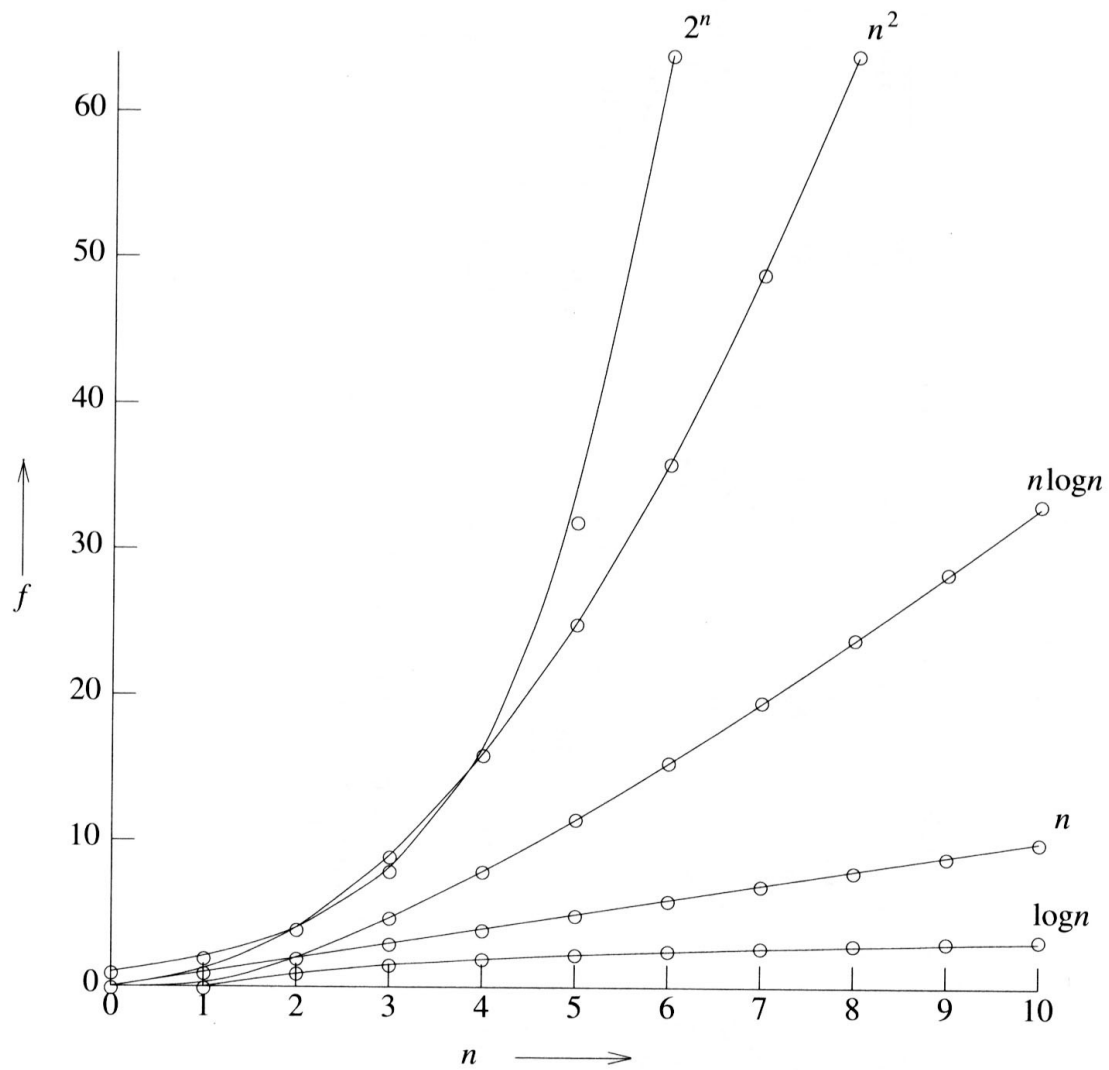


Figure 2 Plot of function values

- The following fig gives the time needed by a 1 billion instructions per second computer to execute a program of complexity $f(n)$ instructions.

Times on a 1 billion instruction per second computer

n	Time for $f(n)$ instructions on a 10^9 instr/sec computer						
	$f(n)=n$	$f(n)=\log_2 n$	$f(n)=n^2$	$f(n)=n^3$	$f(n)=n^4$	$f(n)=n^{10}$	$f(n)=2^n$
10	.01 μ s	.03 μ s	.1 μ s	1 μ s	10 μ s	10sec	1 μ s
20	.02 μ s	.09 μ s	.4 μ s	8 μ s	160 μ s	2.84hr	1ms
30	.03 μ s	.15 μ s	.9 μ s	27 μ s	810 μ s	6.83d	1sec
40	.04 μ s	.21 μ s	1.6 μ s	64 μ s	2.56ms	121.36d	18.3min
50	.05 μ s	.28 μ s	2.5 μ s	125 μ s	6.25ms	3.1yr	13d
100	.10 μ s	.66 μ s	10 μ s	1ms	100ms	3171yr	4*10 ¹³ yr
1,000	1.00 μ s	9.96 μ s	1ms	1sec	16.67min	3.17*10 ¹³ yr	32*10 ²⁸³ yr
10,000	10.00 μ s	130.03 μ s	100ms	16.67min	115.7d	3.17*10 ²³ yr	
100,000	100.00 μ s	1.66ms	10sec	11.57d	3171yr	3.17*10 ³³ yr	
1,000,000	1.00ms	19.92ms	16.67min	31.71yr	3.17*10 ⁷ yr	3.17*10 ⁴³ yr	

μ s = microsecond = 10^{-6} seconds

ms = millisecond = 10^{-3} seconds

sec = seconds

min = minutes

hr = hours

d = days

yr = years

Probabilistic analysis

The Hiring Problem

- You are using an employment agency to hire a new office assistant.
- The agency sends you one candidate each day.
- You interview the candidate and must immediately decide whether or not to hire that person. But if you hire, you must also fire your current office assistant—even if it's someone you have recently hired.
- Cost to interview is c_i per candidate.
- Cost to hire is c_h per candidate.
- You want to have, at all times, the best candidate seen so far.
- When you interview a candidate who is better than your current office assistant, you fire the current assistant and hire the candidate.
- You will always hire the first candidate that you interview.
- Problem: What is the cost of this strategy?

Pseudo-code to Model the Scenario

Hire-Assistant (n)

$best \leftarrow 0$;;Candidate 0 is a least qualified candidate

for $i \leftarrow 1$ **to** n

do interview candidate i

if candidate i is better than candidate $best$

then $best \leftarrow i$

 hire candidate i

Cost Model:

- We are not concerned with the running time of Hire-Assistant.
- We want to determine the **total cost of hiring the best candidate**.
- If **n candidates interviewed** and **m hired**, then **cost is $nc_i + mc_h$** .
- Have to pay nc_i to interview, no matter how many we hire.
- So, focus on analyzing the hiring cost mc_h .
- m varies with order of candidates.

Best-Case Analysis

- Best case is when the agency sends you the best applicant on the first day.
- Cost is $nc_i + c_h$.

Worst-case Analysis

- In the worst case, we hire all n candidates.
- This happens if each candidate is better than all those who came before. Candidates come in increasing order of quality.
- Cost is $nc_i + nc_h$.
- If this happens, we fire the agency.
- What should happen in the average case?

Probabilistic analysis

- Generally, we use probabilistic analysis to *analyze the running time* of an algorithm.
- Sometimes, we can also use it to analyze *quantities* such as the *hiring cost* in procedure Hire-Assistant.

Hiring problem-Average –case Analysis

- An input to the hiring problem is an ordering of the n applicants, there are $n!$ different inputs.
- Average number of candidates hired is equal to

$$\sum_{i=1}^n P(i)$$

- Where $p(i)$ is the probability of i^{th} applicant is better than the previous $i - 1$ applicants.

- Probability of i^{th} applicant is better than the previous $i - 1$ applicants = $1 / i$.
- Suppose $n = 4$ and $i = 3$.

Ex:-Probability that 3rd applicant is better than the 2 previous ones.

1234	2134	3124	4123
1243	2143	3142	4132
1324	2314	3214	4213
1342	2341	3241	4231
1423	2413	3412	4312
1432	2431	3421	4321

$$8/24 = 1/3$$

- Average number of candidates hired is

$$= \sum_{i=1}^n P(i)$$

$$= \sum_{i=1}^n 1/i$$

$$= \ln n + 1$$

Expected hiring cost = $O(c_h \ln n)$.

Amortized Analysis

- Data structures are subjected to a sequence of operations rather than one operation.
- In this sequence, one operation possibly performs certain modifications that have an impact on the run time of the next operation in the sequence.
- One way of assessing the worst case run time of the entire sequence is to add worst case efficiencies for each operation.
- This may result in an excessively large and unrealistic time complexity.

- Amortized analysis concentrates on finding the average complexity of a worst case sequence of operations.
- Amortized analysis considers interdependence between operations.
 - For example, if an array is sorted and only a very few new elements are added, then re-sorting this array will be much faster than sorting it for the first time.

You try to estimate an upper bound of the **total work $T(n)$** required for a sequence of **n** operations...

Some operations may be cheap some may be expensive.
Overall, your algorithm does **$T(n)$ of work for n operations ...**

Therefore, by simple reasoning, the *amortized cost* of each operation is **$T(n)/n$**

Three Methods of Amortized Analysis

- Aggregate analysis:
 - Total cost of n operations/ n ,
- Accounting method:
 - Assign each type of operation an (different) amortized cost
 - overcharge some operations,
 - store the overcharge as credit on specific objects,
 - then use the credit for compensation for some later operations.
- Potential method:
 - Same as accounting method
 - But store the credit as “potential energy” and as a whole.